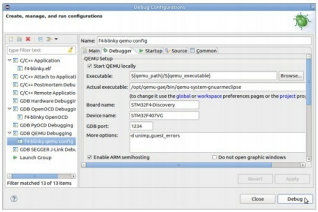

Figure 4: Blinky demo of Qemu debugging settings

QEMU Debugging to generate a new configuration (for the first time only). Under the *Debugger* tab, fill in the Board name as *STM32F4-Discovery* and Device name as *STM32F407VG*, as shown in Figure 4. You can try this with other board names as listed by *qemu-system-gnuarmeclipse -M ?* and other device (MCU cores) names as listed by *qemu-system-gnuarmeclipse -mcu ?*

Now, launch the GDB session by hitting the *Debug* option. The Eclipse perspective will be changed to debug mode and you can try various debug options like *suspend, resume, terminate, step into, step over, step return*, etc. Also, you can watch the state of variables, breakpoints and registers. You can also watch peripheral registers if the *packs* plugin and target-specific packs are installed.

Similarly, if you are flashing the code on a real target, create an OpenOCD configuration and fill in the config options as *-f board/stm32f4discovery.cfg* under the Debugger tab, and launch an OpenOCD session. You may rename the generated configuration in a convenient way. You can specify a suitable OpenOCD script name for other targets under the config options.

Once the debug session is launched, you will observe that the selected LED is blinking for a given time interval on the real target or on the graphical window enabled by Qemu emulation, and semi-hosting output from the target under the *Console* tab. At present, the Qemu based simulator supports LED blinking and button-pressing only; to test other peripherals, you need to go for a real target.

If you are not planning to trace the code, step by step, or watch the intermediate snapshots, right click on the project and select *Run Configurations* under the *Run As* sub-menu. This executes code directly on a real target or under Qemu. Run and debug sessions can also be launched using the available menu options or toolbar options.

RISC-V architecture support

RISC-V, pronounced as RISC five, is an open instruction set architecture that is based on RISC principles. It enables open hardware design in a better way as the ISA is patent-free, unlike other architectures, and many CPU designs are available under the BSD licence. Several projects based on the RISC-V architecture are hosted at *github.com/riscv* like the toolchain, debugger, emulator and Linux kernel port, file system, standard libraries, etc. The GNU MCU Eclipse project comes with added support for this open architecture through the plugins of v4.x onwards.

Since the project code, necessary libraries, toolchain and plugins are open under this environment, you can choose the desired versions of the toolchain and debugger from any repository of your choice. This helps in getting a better understanding of the whole development cycle and the anatomy of each phase, as well as the generated outcome. You can try these steps and explore further! END

Reference

https://gnu-mcu-eclipse.github.io/en.wikipedia.org/wiki/RISC-V

By: Rajesh Sola

The author is a faculty member of C-DAC's Advanced Computing Training School, and an evangelist in the embedded systems and IoT domains. You can reach him at *rajeshsola@gmail.com*.